

Documenting C++ with Doxygen and Sphinx - doxyrest

Rohit Goswami · 2026-02-02 · documentation · workflow · pixi · cpp

Unifying C++ and Python documentation.

Background

For almost half a decade my C++ projects have been either scoped to where plain Doxygen works well enough [^fn:1], or largely undocumented at the C++ API level [^fn:2]. While integrating with Sphinx via `exhale` or `breathe` shown earlier [^fn:3] works well, it often fails to provide a smooth source code view or struggles with modern C++ constructs.

For smaller, well formed libraries, the drawbacks of not being able to read the source are manifold, tending towards over-verbose documentation strings. Back in 2020 during the [season of docs for Symengine](#), I stuck to pure `doxygen` with a [custom theme](#), `doxyYoda`, a setup which served me well for Fortran projects like `GaussJacobiQuad` [^fn:4], and continues to be attractive.

Given the advent of `pixi`, however, it is worth revisiting the `doxyrest` setup [^fn:5]. For the most part, the focus will be on implementation within a `meson` based project, `rgpot`, which defines an RPC interface for accessing forces and energies, typically for an atomistic calculation, before demonstrating the benefit for a much larger project, `metatomic`.

****Note****

For before/after images see: - [rgpot (gh-25)](<https://github.com/OmniPotentRPC/rgpot/pull/25>) - [metatomic (gh-148)](<https://github.com/metatensor/metatomic/pull/148>)

Baseline – `rgpot`

In general, the pipeline relies on Doxygen emitting XML, which `doxyrest` parses via Lua templates to generate reStructuredText, finally consumed by Sphinx. This allows the C++ API to sit naturally alongside Python documentation in a single Sphinx project, utilizing modern themes like `furo` or `pydata-sphinx-theme`.

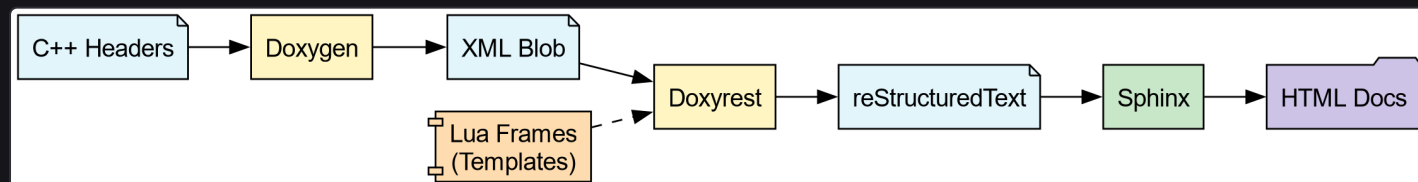


Figure 1: Overview of the 'doxyrest' build process

The `pixi` task workflow

For `rgpot`, I opted to encapsulate the build complexity within `pixi` tasks. This avoids polluting the repository with git submodules for tooling that is essentially a build-time dependency, and has better support for managing dependencies.

The workflow is broken down into dependency retrieval, patching, and generation:

```
[feature.docs.tasks.setup-doxyrest]
description = "Clones the doxyrest dependency into the subprojects directory"
cmd = """
test -d subprojects/doxyrest || \
(mkdir -p subprojects && \
git clone --depth 1 --single-branch \
https://github.com/vovkos/doxyrest subprojects/doxyrest)
"""
```

The `setup-doxyrest` task handles the “shallow clone” approach to keep the checkout light.

The Build Cycle

With the infrastructure in place, the actual build is a two-step process defined in `doxybuild`:

```
[feature.docs.tasks.doxybuild]
description = "Generates XML via Doxygen and converts it to RST via Doxyrest"
cwd = "docs/source"
depends-on = ["setup-doxyrest"]
cmd = "doxygen Doxyfile_potlib.cfg && doxyrest -c doxyrest-config.lua"
```

We first generate the XML blobs with Doxygen. Immediately after, `doxyrest` is invoked with a Lua configuration file, which defines the files used to map the generated XML into frames. Internally, `doxyrest` relies on **Lua Frames** which are templates that mix standard `reStructuredText` with injected Lua logic (delimited by `%{ ... }`) and variable substitution (via `$var`).

In principle, one could write custom frames to radically alter the output, for C++ projects we simply point `FRAME_DIR_LIST` to the standard “C-family” frames provided by the repo we cloned earlier.

```
INPUT_FILE = "xml/index.xml"
OUTPUT_FILE = "api/index.rst"
INDEX_TITLE = "API Reference"

FRAME_DIR_LIST = {
    "../../subprojects/doxyrest/frame/common",
    "../../subprojects/doxyrest/frame/cfamily"
}

LANGUAGE = "cpp"
ESCAPE_ASTERISKS = true
ESCAPE_TRAILING_UNDERSCORES = true
```

With this, `doxyrest` digests the Doxygen XML into produce RST files that Sphinx can ingest.

Sphinx Configuration

For `Sphinx` to process the generated RST correctly, it needs to load the `doxyrest` extension. Since we are downloading/cloning this into a local subdirectory, we must explicitly update the python path in `docs/src/conf.py` so Sphinx can find it.

```
# For doxyrest, the location of this can change
sys.path.insert(0, os.path.abspath("subprojects/doxyrest/sphinx"))

extensions = [
    # ... other extensions
    # c++ helpers
    "doxyrest",
    "cpplexer",
    # ...
]
```

Finally, the standard Sphinx build can pick up the artifacts:

```
[feature.docs.tasks.sphinxbld]
depends-on = [ { task = "mkrst", environment = "docs" }, ]
cmd = """ sphinx-build docs/source/ docs/build """
```

CI deployment

The primary advantage of encapsulating the build logic within `pixi` tasks lies in the simplification of Continuous Integration (CI) pipelines. Since the environment and build steps reside within `pixi.toml`, the CI configuration need only setup `pixi` and invoke the target task.

```
- uses: prefix-dev/setup-pixi@v0.8.10
  with:
    activate-environment: true
    environments: docs
- name: Generate Docs
  run: pixi r docbld
```

This reduces the friction of maintenance significantly, as local reproduction of CI failures becomes a matter of running `pixi r docbld`, and integrates easily with other actions, like the [doc-previewer](#) to post artifact links directly on Pull Requests. This helps with easier review of the generated output without digging through Action logs.

Metatomic integration

For larger projects like [metatomic](#) which do not use `pixi`, the process requires a bit more manual orchestration. Here, the challenge is replacing the declarative task runner with a reliable script that ensures dependencies (like the `doxyrest` `lua` frames) are present before invoking the build.

While a `Makefile` or `bash` script is the traditional route, using [nushell](#) provides a nice balance of readability and cross-platform handling without the foot-guns of strict POSIX shell scripting, which are best avoided, despite my [return to bash](#).

The ideal orchestrator script

Instead of relying solely on `tox` and file driven configurations to do the heavy lifting of environment setup, we use a `nu` script to bootstrap the documentation build.

First, we define our environment and ensure the local directory structure exists. We check if `doxyrest` is available in the path (useful for local development) or if we need to fetch it.

```

let root = ($env.PWD)
let deps_dir = ($root | path join "docs/subprojects")
let doxyrest_src = ($deps_dir | path join "doxyrest")
let install_dir = ($deps_dir | path join "install")
let has_doxyrest = (which doxyrest | is-not-empty)

if not ($deps_dir | path exists) { mkdir $deps_dir }

```

Next, if the binary is missing, we download the pre-compiled release. This avoids the complexity of setting up a C++ build chain in the CI environment just for a doc tool.

```

if not $has_doxyrest and not ($install_dir | path exists) {
  print "Downloading Doxyrest binary release..."
  let version = "2.1.3"
  let target = "linux-amd64"
  let archive = $"doxyrest-($version)-($target).tar.xz"
  let url = $"https://github.com/vovkos/doxyrest/releases/download/doxyrest-($version)/($archive)"

  cd $deps_dir
  http get $url | save $archive
  tar -xJf $archive

  if ($install_dir | path exists) { rm -rf $install_dir }
  mv $"doxyrest-($version)-($target)" $install_dir
  rm $archive
  cd $root
}

```

Even with the binary, we still need the Lua frames (themes/templates) from the source repository. We clone this shallowly.

```

if not ($doxyrest_src | path exists) {
  print $"Cloning doxyrest frames into ($doxyrest_src)..."
  git clone --depth 1 --single-branch https://github.com/vovkos/doxyrest $doxyrest_src
}

```

Finally, we run the pipeline. Note that we prepend the local install directory to the `PATH` only for the `doxyrest` execution block.

```

print "Generating XML with Doxygen..."
cd docs
if not ("build/doxygen/xml" | path exists) { mkdir build/doxygen/xml }
doxygen Doxyfile.metatomic

print "Generating RST with local Doxyrest..."
with-env { PATH: ($env.PATH | prepend ($install_dir | path join "bin")) } {
  doxyrest -c doxyrest-config.lua
}

print "Building HTML with Sphinx (tox)..."
tox -e docs

```

Sphinx and CI configurations are similar to the previous project, with the complete including a `bash` orchestrator in the [PR here](#).

► Click to see the full `scripts/mkdoc.nu` script

Conclusions

Personally, I like the look of Doxygen still, and for some themes like [shibuya](#), manual patches are required (e.g. [monkeypatch](#) @ [rgpot](#)). Nevertheless there are multi-language projects [\[^fn:6\]](#) which benefit from having a consistent style across languages, so this works out quite well. Probably the next step would consistency across even more languages and esoteric builds, like those in [Metatensor](#).

```
computational chemists don't write C++ anyway, the user-docs and YAML are
enough \[^fn:3\]: or in \[Chemfiles\]\(https://chemfiles.org/chemfiles/0.10.4/classes/frame.html\) \[^fn:4\]: before
\[FORD\]\(https://forddocs.readthedocs.io/en/stable/\) became popular
as well \[^fn:5\]: a fantastic, if slightly under-documented code \[^fn:6\]: like \[metatomic\]
\(https://docs.metatensor.org/metatomic/latest/index.html\)
```